



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks

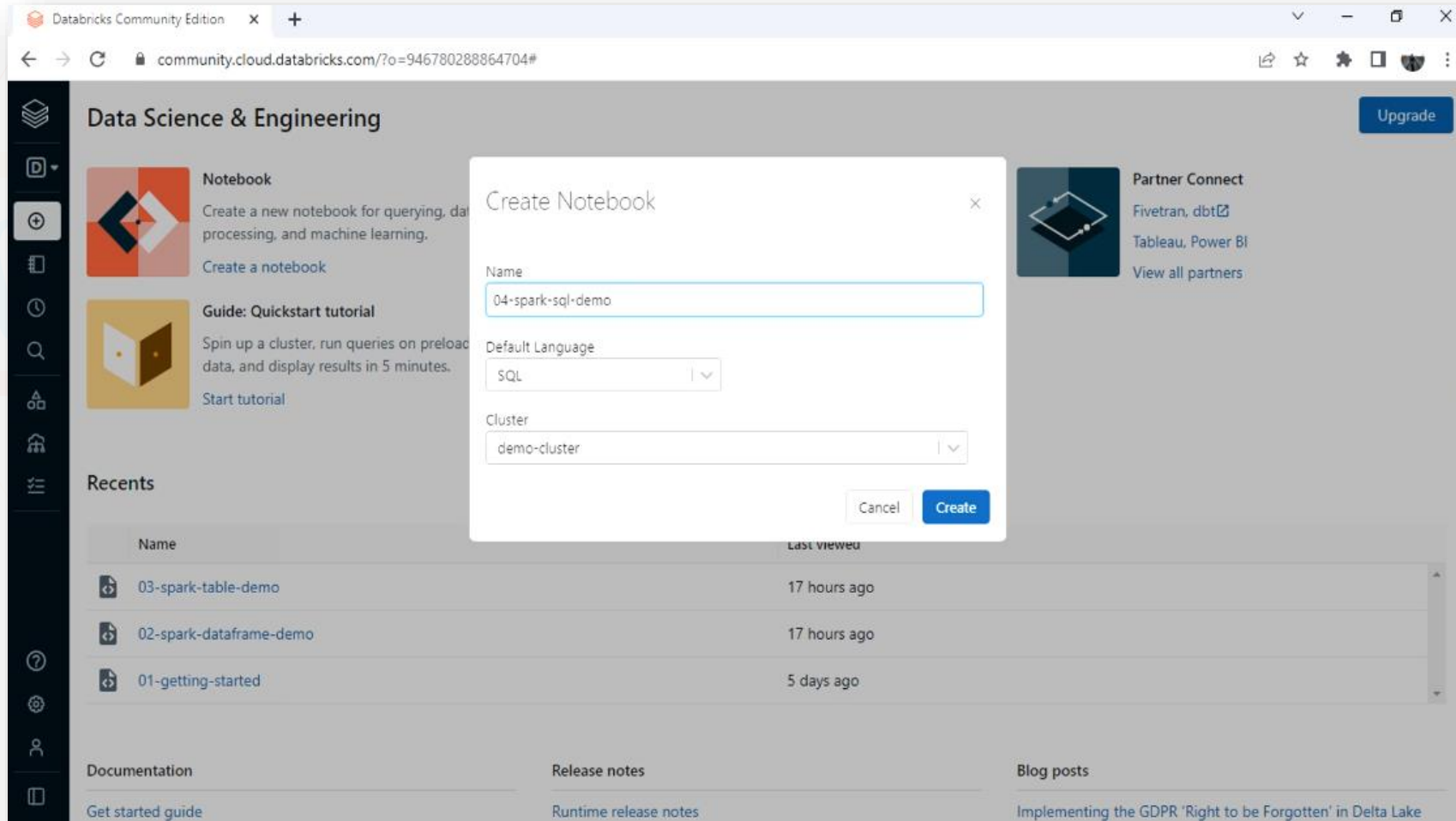


Data Caching Essentials

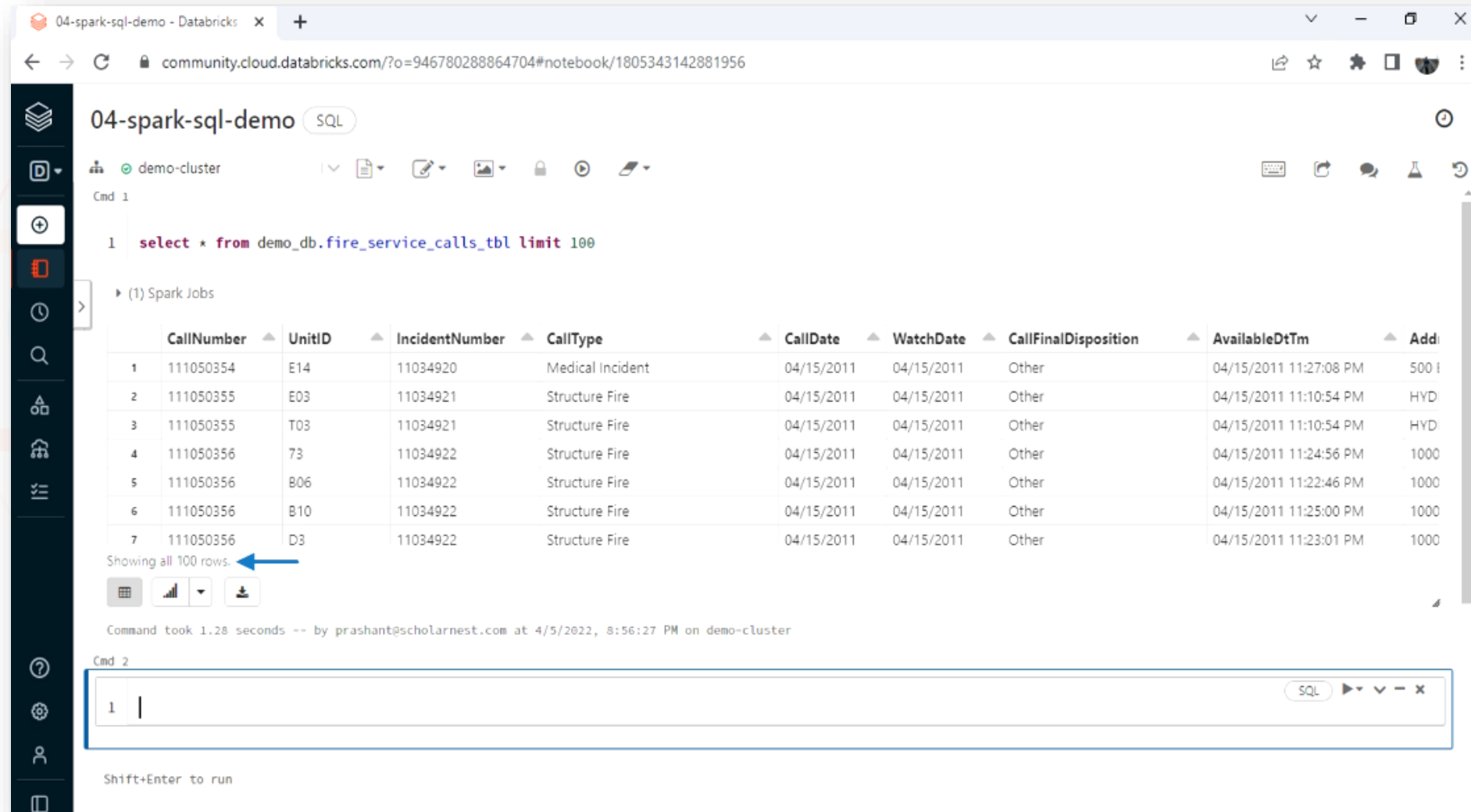
Go to your Databricks workspace.

Create a notebook. **(Reference: 04-spark-sql-demo.sql)**

We want to use Spark SQL for data analysis. So choose SQL as the default language of your notebook. Make sure to attach a cluster to your notebook.



We created a table for the San Fransisco fire call response dataset. Now let me type the code to select hundred records from the table.



The screenshot shows a Databricks notebook interface. The browser address bar indicates the URL: `community.cloud.databricks.com/?o=946780288864704#notebook/1805343142881956`. The notebook title is "04-spark-sql-demo" and the language is set to "SQL". The cluster is named "demo-cluster".

Under "Cmd 1", the following SQL query is entered:

```
1 select * from demo_db.fire_service_calls_tbl limit 100
```

Below the query, it says "(1) Spark Jobs". The results are displayed as a table with 10 columns: CallNumber, UnitID, IncidentNumber, CallType, CallDate, WatchDate, CallFinalDisposition, AvailableDtTm, and Address. The first 7 rows are visible, showing various fire incidents from April 15, 2011.

	CallNumber	UnitID	IncidentNumber	CallType	CallDate	WatchDate	CallFinalDisposition	AvailableDtTm	Address
1	111050354	E14	11034920	Medical Incident	04/15/2011	04/15/2011	Other	04/15/2011 11:27:08 PM	500 I
2	111050355	E03	11034921	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:10:54 PM	HYD
3	111050355	T03	11034921	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:10:54 PM	HYD
4	111050356	73	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:24:56 PM	1000
5	111050356	B06	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:22:46 PM	1000
6	111050356	B10	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:25:00 PM	1000
7	111050356	D3	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:23:01 PM	1000

Below the table, it says "Showing all 100 rows." with a blue arrow pointing to the text. There are icons for table, chart, and download. The command execution time is shown as "Command took 1.28 seconds -- by prashant@scholarnest.com at 4/5/2022, 8:56:27 PM on demo-cluster".

Under "Cmd 2", there is an empty text box for entering a new command. The language is still set to "SQL".

I want to cache this table into memory to speed up my queries. I want to run ten queries on the same table. So it makes sense that I cache this table into the memory and avoid reading data from the disk for every query. And we have the code for the same in the snapshot below.

The screenshot shows a Databricks SQL notebook interface. The top section displays a table with 10 columns and 4 visible rows of fire service call data. Below the table, the command history shows two executed SQL commands. The first command caches the table and performs a select query. The second command is partially entered in the input field.

id	call_id	unit_id	location	type	date	time	category	timestamp	count
4	111050356	73	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:24:56 PM	1000
5	111050356	B06	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:22:46 PM	1000
6	111050356	B10	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:25:00 PM	1000
7	111050356	D3	11034922	Structure Fire	04/15/2011	04/15/2011	Other	04/15/2011 11:23:01 PM	1000

Showing all 100 rows.

Command took 1.28 seconds -- by prashant@scholarnest.com at 4/5/2022, 8:56:27 PM on demo-cluster

Cmd 2

```
1 cache lazy table fire_service_calls_tbl_cache as
2 select * from demo_db.fire_service_calls_tbl
```

OK

Command took 0.40 seconds -- by prashant@scholarnest.com at 4/5/2022, 8:58:52 PM on demo-cluster

Cmd 3

```
1 |
```

Shift+Enter to run

You must learn the following things about the Spark SQL cache table.

1. How does the Spark cache table works?

The cache-table-as command will create a temporary view using the Select expression. So you already have a table. But when you cache it, you will create a temporary view on the table. Once your table is cached as a temporary view, you can use the view or the table. Once cached, Spark will always bring data from the cache.

2. How to make it re-executable?

The cache-table-as command creates a temporary view, so you cannot rerun it. And if you try re-running the cache command again, you will see a “temporary view already exists” error. You can fix this issue by dropping the temporary view before rerunning the cache-table-as command.

3. What is Lazy?

The lazy is to make sure we do not cache the table until it is used. It is optional. You can also remove it, but it is recommended to keep it as the execution will be very fast because it simply marks the table to be cached on its first use.

Caching the table consumes a lot of memory. So you should make use of this feature carefully. We have some guidelines:

1. Do not cache tiny tables because tiny table will anyway work faster.
2. Do not cache huge tables that cannot fit in the Spark memory because there is no point in caching a table into memory if it cannot fit in the memory.

It is recommended to cache medium-sized frequently used tables.

So, you should cache a table if you meet the following criteria:

1. Table data is a reasonable size and fits into the memory.
2. Table is frequently used. You should not cache a table that you do not want to use again and again.



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com